Ders 4 Genişletilmiş Notu: Hayat Bilgisi Projeleri, Kelime Benzerliği ve Saklı Anlam İndeksleme

Ders 4 Genişletilmiş Notu: Hayat Bilgisi Projeleri, Kelime Benzerliği ve Saklı Anlam İndeksleme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ödev Sunumlarının Ortak Zemini: Hayat Bilgisi Veri Tabanı
- 2. Graf Veri Tabanıyla Kelimeler Arası Yol Bulma
- 3. Regex ve word2vec ile Bilgi Çıkarımı
- 4. Hayat Bilgisiyle Cümle Zenginleştirme
- 5. Kullanıcı Geri Bildirimiyle İlişki Doğrulama
- 6. Doğruluk ve Kapsayıcılık Değerlendirmesi
- 7. Kelimeleri İki Boyutta Görselleştirme
- 8. FastText ile Kelime Temsili ve Analojiler
- 9. Geçen Dersin Tekrarı: İstatistiksel Dil Modelleri
- 10. Ardışık Geçme Yerine Aynı Dokümanda Geçme
- 11. Kelime Benzerliği: Karakter ve Anlam
- 12. Saklı Anlam İndeksleme (LSI)
- 13. Sonraki Konular: Kavramsal Ağaç ve Anlamsal Uzaylar
- Örnek Sorular
- Dersin Toparlaması

1. Ödev Sunumlarının Ortak Zemini: Hayat Bilgisi Veri Tabanı

Dersin ilk yaklaşık bir buçuk saati birinci ödevin proje sunumlarına ayrıldı. Bütün projelerin girdisi aynıydı: sınıfın önceki haftalarda doldurduğu Türkçe hayat bilgisi (common sense) veri tabanı. Bu veri tabanında yaklaşık 500 isim ve 500 fiil için çeşitli sorulara verilmiş kısa cevaplar vardır. İsimler için "kim kullanır, nerede bulunur, ham maddesi nedir, içinde neler bulunur, rengi nedir, canlı mı cansız mı, ağırlığı nedir"; fiiller için "kimle neyle yapılır, neye kime yapılır, nasıl yapılır, nerede yapılır, ne olunca yapılır, yapınca ne olur, fiziksel mi zihinsel mi" gibi sorular sorulmuştur. Veri öğrencilere Prolog (`.pl`) dosyaları olarak verilmişti.

Bu bölümde sunulan projeler adsız olarak, yalnız akademik içerikleriyle aktarılıyor. Sunumların ortak gözlemi, verinin bir kısmının gürültülü olmasıydı: bazı cevaplar soruyla ilgisizdi, bazıları sözlükten kopyalanmış gibiydi, Türkçe karakterlerde kodlama sorunları vardı. Hoca bu konuda şunu söyledi:

> **Hocanın vurgusu:** Gerçek veri kümeleri böyledir, hiçbir şey hayatta altın tepside sunulmaz. Hatta bu veri kümesi, bu tür işlerle uğraşmış olanların takdir edeceği gibi, çok temiz bir veri kümesidir.

2. Graf Veri Tabanıyla Kelimeler Arası Yol Bulma

Bir grup, kelimeler arasındaki bağlamsal ilişkileri bir graf veri tabanında aradı. Kullanılan araç Neo4j'dir. Neo4j, SQL'e benzeyen Cypher adlı bir sorgu diline sahip bir graf veri tabanı-

dır. Düğümler arasında yönlü ya da yönsüz yol bulma, ilişki arama ve sonucu şekil üzerinde gösterme imkânı verdiği için seçildi.

Yapılan işlemler şunlardır (ekrandan, 00:20:00):

- Veri “kelime, ilişki türü, cevap” biçiminde düzenlendi.
- Türkçe karakterler düzeltilerek kelimeler tekilleştirildi (aynı kelimenin farklı yazımları birleştirildi).
- Tek bir düğüm türü üzerinden `verb`, `noun` ve `answer` özellikleri kullanılarak kelimeler arasında ilişki kuruldu.
- Python arayüzünde kullanıcıdan iki kelime, sorgu sayısı ve ilişki sayısı (yol uzunluğu) alınır.

Örneğin “odun” ile “araba” arasında altı ilişkilik yollar istendiğinde bir yol şöyle çıktı: odun, “neye kime yapılır” ile kesmek; kesmek ile kamp yapmak “fiziksel” ortak özelliği üzerinden; kamp yapmak, “kimle neyle yapılır” ile ateş; ateş ile ısınmak; ısınmaktan da araba. Grup, böyle bir yolun bir tür profil çıkarımı da sağladığını söyledi (odun ve araba birlikte “kampa gitmek” gibi bir etkinliği akla getiriyor). Hoca, iki kelime arasında belirli uzunlukta ve belirli ilişki türleri içeren bir yol bulmanın arka taraftaki graf veri tabanının hazır bir özelliği olduğunu teyit ettirdi; grup hangi ilişki türlerinin yolda kullanılacağını kendilerinin belirttiğini söyledi.

Yüklenen grafta yaklaşık 38.777 düğüm ve 125 bin civarında ilişki vardı. Bazı ilişkiler kendi üzerine dönüyordu. Verideki anlamsız cevaplar yolları bozdu: “şofben kim kullanır” sorusuna “sarı”, “ateş etmek nasıl yapılır” sorusuna “kırmızı” cevabı verilmişti. Yönsüz aramada ise yollar aynı kalıpları tekrarladı, çünkü arama en kısa düğümler üzerinden ilerliyor ve ilk 10 sonuç gösteriliyordu. Bazı sorgular 5000 sonuç döndürüyordu; bunların arasından anlamlı olanları seçmek ayrı bir problem olarak kaldı.

Canlı denemede sınıftan alakasız iki kelime istendi (“mektup” ve “armut”). Sistem armut, canlı/cansız, pul, mektup üzerinden bir yol buldu. Daha uzun yollar için ilişki sayısının artırılabileceği gösterildi.

Grup, ileriki iş olarak cümle düzeyine çıkmayı önerdi. Örneğin “Ali eve gitti” ve “Ali Akbil kullandı” cümlelerinden Ali’nin otobüse bindiği, belki arabasının olmadığı gibi çıkarımlar yapılıp kişi profili çıkarılabilir; bu da kişiye uygun reklam gibi uygulamalara götürebilir.

3. Regex ve word2vec ile Bilgi Çıkarımı

Bir öğrenci, bilgi çıkarımı üzerine çalıştı. Geliştirme ortamı PyCharm ve Python 3.7; kullanılan kütüphaneler NumPy , Pandas, regex, Gensim, NLTK, SQLAlchemy ve arayüz için Streamlit’tir. Amaç, hayat bilgisi verisinden bir veri tabanı oluşturup bu veri tabanına ve regex şablonlarına dayalı bir bilgi çıkarımı modülü kurmaktı. Sistem bir metin alır ve cümlelerdeki “nasıl, ne yaptı, nerede, ne zaman, neden, ne olur, kim” sorularının cevaplarını çıkarmaya çalışır.

3.1 Sözlük tablolarının kurulması

İsim ve fiil dosyaları okunup iki tablo (isimler ve fiiller ilişkileri) oluşturuldu. Bunlardan sekiz sözlük tablosu türetildi:

- Sıfatlar: isimlerin renk, şekil gibi sıfat bilgileri ve fiillerin “nasıl yapılır” cevapları.
- Fiiller: fiillerin kendisi.
- Ham madde: isimlerin “ham maddesi” ve “içinde neler bulunur” cevapları.
- Yer: isimlerin “nerede bulunur”, fiillerin “nerede yapılır” cevapları.
- Kim: isimlerin “kim kullanır”, fiillerin “kim ne yapar” cevapları.
- Ne olur: fiillerin “yapınca ne olur” cevapları.
- Ne zaman: fiillerin “ne olunca yapılır” cevapları.
- Neden: fiillerin “niçin yapılır” cevapları.

3.2 Çıkarım akışı

Akış şöyledir (ekrandan, 00:40:00):

```
METİN -> cümlelere böl
      -> Türkçe karakter dönüşümü ve küçük harfe çevirme
      -> veri tabanında (vocabulary.db) metnin fiillerini ara
         bulundu mu? EVET -> bulunanları dön
                     HAYIR -> regex şablonunu çalıştır
                              bulundu mu? EVET -> bulunanları dön
                                           HAYIR -> veri tabanındaki kelimeleri
                                                    word2vec benzerliğine göre al
                                                    -> bulunanları dön
```

Fiil için şablon, fiil çekim eklerini yakalar; bulunan yeni fiiller sözlüğe eklenerek veri tabanı büyütülür. Öteki sorular için şablonlar Türkçe eklere dayanır: “nerede” için bulunma ve ayrılma hâli ekleri; “ne zaman” için “-dığında, -dığı zaman, -yorken”; “neden” için “-dığı için, -dan dolayı, -dan ötürü” ve bunlardan önceki kelime.

Son adımda word2vec kullanılırken Türkçe karakter dönüşümü yapılmaz, çünkü kullanılan word2vec modeli Türkçe karakterlerle eğitilmiştir; yalnız stop word’ler atılır ve küçük harfe çevrilir. Kosinüs benzerliği seçilmişse sözlükteki kelimelerle cümledeki kelimeler arasındaki kosinüs benzerliğine, seçilmemişse word2vec’in en benzer kelime yöntemine bakılır. Veri tabanında örneğin sıfatlar tablosunda gerçekte sıfat olmayan kelimeler de bulunduğu için çok anlamsız eşleşmeler çıkabiliyordu. Bunları elemek için hepsi sıfat olan küçük bir varsayılan liste kullanıldı.

3.3 Sonuçlar ve yorum

Demoda “nasıl” sorusuna “soğuk, minik” gibi uygun cevaplar, “ne yaptı” sorusuna çoğunlukla doğru fiiller, “nerede” sorusuna “evden, yolda, yerde” gibi ilgili kelimeler bulundu. En kötü sonuç “kim” sorusundaydı; ilgili tabloya çok fazla fiil karışmıştı. Hoca, sistemin cevaplarının tek kelimelik olduğunu, “şunu şu sebeple yaptı” gibi ikili ilişkiler çıkarmadığını teyit ettirdi ve şöyle yorumladı: sistem aslında cümlenin öğelerini buluyor, sonra her öğeye dair bulduğu tüm adayları listeliyor.

Önerilen geliştirmeler: veri setini iyileştirmek, sözlükte birebir eşleşme yerine edit distance gibi birkaç harf farkı yakalayan yöntemler kullanmak, öğeleri ayırmak için POS etiketleyici kullanmak. Öğrencinin asıl hedefi günlük haber metinlerinden yapılandırılmış veri çıkarmaktı.

4. Hayat Bilgisiyle Cümle Zenginleştirme

Bir grup, kullanıcıdan alınan bir cümleden yeni cümleler üretmek (data augmentation) üzerine çalıştı. Cümledeki isim ve fiillerin hayat bilgisindeki ilişkileri kullanılarak cümleye yeni ifadeler eklenir. İkinci amaç, ilişkilerin doğruluğunu kullanıcı geri bildirimiyle artırıp azaltmaktır.

Algoritma adımları:

- İsim ve fiil `.pl` dosyaları okunup sözlük (dictionary) yapısında tutulur.
- İlişkili kelime iki yolla seçilir: rastgele, ya da ilişkilerde en sık girilen cevaplara göre (frekanslar bir veri tabanında tutulur).
- Kullanıcının cümlesi Zemberek ile POS etiketlerine ayrılır; isim ve fiiller morfolojik olarak kök ve eklerine ayrılır.
- İsim kökleri isim hayat bilgisinde, fiil kökleri fiil hayat bilgisinde aranır; eşleşen ilişkiler alınır.
- Seçilen ilişkili kelimeler uygun kelimeciklerle (ör. “ile”, “-de/-da”, “şeklinde”) cümleye eklenir.

Kullanıcı ayrıca eklemek istediği ilişki sayısını girer. Hoca bunun ne demek olduğunu sordu. Cevap: üretilen cümle sayısı değişmez; ilişki sayısı kadar sütun (ör. “nerede”, “nasıl”) seçilir ve bu ilişkilerin cevapları tek bir cümleye eklenir. Konsoldaki örnek çıktı şöyledir (ekrandan, 00:50:00):

```
Lütfen bir cümle giriniz:
ali kahve içti
Lütfen eklemek istediğiniz ilişki sayısını giriniz:
2
```

```
Analysis 2: [kahve:Noun] kahve:Noun+A3sg
Analysis 3: [içmek:Verb] iç:Verb+ti:Past+A3sg
ali { bira ile } kahve içti
ali { masa de/da } kahve içti
ali { toz şeklinde } kahve içti
ali   bira ile , masa de/da , toz şeklinde , kahve içti
```

Hoca yöntemi şöyle özetledi: var olan kelimeler başka kelimelerle değiştirilmiyor, cümleye uygun olabilecek ek ifadeler eklenerek cümle uzatılıyor. Bütün eklemelerin tek cümlede toplanmasının komik sonuçlar verebileceğini, her birini ayrı cümle olarak üretmenin belki daha iyi olacağını söyledi.

Frekansa dayalı sürümde "resim çizdi" gibi bir cümleden "evde resim çizdi", "konuşarak resim çizdi", "resim çizdi ve mutlu oldu" gibi cümleler üretilir. Kullanıcı mantıklı cümleleri işaretleyince ilgili ilişkinin sayacı bir artırılır, mantıksız dediklerinde azaltılır. Grup, bunun gönüllü kullanıcılarla ya da oyun biçiminde yapılabileceğini, kök ve ek işlemleri üzerinde daha çok çalışılarak daha anlamlı cümleler üretilebileceğini söyledi. Zemberek Python içinden çağrıldı.

5. Kullanıcı Geri Bildirimiyle İlişki Doğrulama

5.1 Görsellerle doğrulama

Bir öğrenci, veri tabanındaki cevapların güvenilirliğini ölçmek ve veri tabanına yeni bilgi eklemek için görsellere dayalı bir arayüz yaptı. Her soru-cevap çifti için arama motorundan cevaba ait görseller önceden indirildi. Arayüz (Tkinter) soruyu ve görseli gösterir; kullanıcıya kendi aklındaki cevabın görselle ilişkili olup olmadığı sorulur.

- "Evet": var olan bilgi teyit edilir.
- "Hayır": kullanıcı kendi cevabını yazar; bu cevap veri setine yeni bilgi olarak eklenir.
- "Fikrim yok": kayıt yapılmaz.

Hoca, ekrandaki görsellerin ne olduğunu sordu; cevap, örneğin "acele etmek fiziksel mi zihinsel mi" sorusunda verilen "fiziksel" cevabına karşılık gelen görseller olduğuydu. Demoda bazı görseller alakasız çıktı. Örneğin "canlı/cansız" sorusunda bir dizi karakterinin fotoğrafı geldi, çünkü karakterin adı arama terimiyle aynıydı. Türkçe karakter dönüşümü yüzünden "taş" kelimesi "tas" olunca bir tas görseli geldi. Bazı tanım cevapları sözlükten birebir kopyalanmış gibiydi; hoca bunu "TDK'dan kopya çekenler olmuş" diye özetledi.

Yetişmeyen kısım: aynı soruya verilmiş farklı cevapların görsellerini birlikte gösterip kullanıcıdan en ilgili olanları seçmesini istemek, seçilen cevapların sıralama (ranking) değerini artırmak. Böylece çok kullanıcıdan gelen oylarla her ilişki için bir güvenilirlik derecesi oluşacaktı. Öğrenci, fikrin kullanıcı katkısıyla öğrenen başka uygulamalardan (Türkçe deyim öğrenen bir sohbet robotu ve hocanın daha önce yaptığı bir hayat bilgisi toplama uygulaması) geldiğini söyledi.

5.2 Sayaçlı güven seviyesi

Bir başka öğrenci, veriyi metin dosyalarına çevirip C# ile bir masaüstü uygulaması yazdı. Kullanıcı fiillerden ya da isimlerden seçer; uygulama rastgele bir ilişki getirir ve doğru olup olmadığını sorar. Doğruysa ilişkinin sayacı artırılır, değilse azaltılır. "Senkronize et" ile farklı kullanıcıların kayıtları birleştirilir, "raporları al" ile ilişkiler sayaçlarıyla listelenir.

Soru: Var olan ilişkileri kullanıcıya gösterip, kaç kişinin olumlu ya da olumsuz işaretlediğini arka tarafta mı tutuyorsun? **Cevap:** Olumlu için +1, olumsuz için -1 ekleniyor. Bu yüzden kaç kişi bilgisi çıkmayabilir: üç kişi evet, üç kişi hayır derse sonuç yine sıfır olur.

6. Doğruluk ve Kapsayıcılık Değerlendirmesi

Bir öğrenci, derste hayat bilgisi veri tabanlarının değerlendirilmesinde geçen iki ölçütü veri tabanımıza uyguladı: doğruluk ve kapsayıcılık.

6.1 Doğruluk

Slayttaki "100 tane veri seçip bir kişinin doğru olarak etiketlemesi" fikrinden yola çıkıldı. İsimler ve fiiller dosyalarından Python'un `random` kütüphanesiyle rastgele 100'er ilişki seçildi. Öğrenci bunları elle kontrol edip doğru olanların yanına + koydu, sonra regex ile + olanları saydırdı. Sonuç: isimlerde 100 ilişkiden 87'si (%87), fiillerde %96'sı doğru.

6.2 Kapsayıcılık: kavram sayısı

Kavram sayısı için başka kaynaklarla karşılaştırma yapıldı. Türkçe için kullanılabilecek kaynak olarak Zemberek, İngilizce için WordNet seçildi.

- WordNet'te 82.115 isim ve 13.767 fiil bulunuyor. Veri tabanındaki isimler Google Translate API ile İngilizceye çevrildi ve WordNet'te arandı; 486 isim bulundu. Fiillerde 468 eşleşme oldu. Fiillerde lemmatization uygulanarak kök hâli arandı. Çeviri hataları ve çevrilemeyen kelimeler bu sayıları biraz düşürdü.
- WordNet'teki eş anlam (synonym), zıt anlam (antonym), alt kavram (hyponym) ve üst kavram (hypernym) ilişkileri bizim ilişki tablomuzda yoktu; özellikle zıt anlam hiç yoktu. Bunları eklemek için eş ve zıt anlamlılar WordNet'ten çekildi, sonra Türkçeye geri çevrildi (ekrandan, 01:20:00). Örnekler: "date" için `day_of_the_month`, "bag" için `travelling_bag`; geri çeviride "broşür" için eş anlam "kitapçık", "çalgı" (İngilizce üzerinden "enstrüman") için "aracı". Zıt anlamlıların çoğu boş geldiği için ve eksik değerlerle dolu bir veri seti istenmediği için bunlar veri setine eklenmedi.
- Zemberek'te 28.884 kelimelik bir ana sözlük dosyası incelendi; 492 isim ve 420 fiil eşleşti. Eşleşme için Zeyrek kütüphanesiyle lemmatization yapıldı (ör. veri tabanında "rahatsız etmek" varken Zemberek'te "rahatsız" vardı).

6.3 Kapsayıcılık: ilişki sayısı

Slayttaki "bir kavram hakkında bir dakika içinde ilişki üretin" fikri üç kişiyle uygulandı. "Çanta" kavramı için akla gelen bütün ilişkiler bir dakikada yazıldı. Birinci kişinin yazdığı 12 ilişkiden 8'i veri setinde vardı (yaklaşık %66). İkinci kişide oran %80, üçüncüde %71 oldu; ortalama yaklaşık %72. Sonuç: veri tabanı bir kişinin üretebileceği ilişkilerin yaklaşık %72'sini kapsıyor.

Soru: Kaç kavram üzerinde çalıştırdın, yalnız çanta için mi? **Cevap:** Evet, yalnız çanta için; daha fazlası yapılabilirdi.

Soru: Kişilere hangi tür ilişkileri üretmeleri gerektiğini söyledin mi? **Cevap:** Biraz bahsedildi ("nerede, nasıl, ağırlık" gibi). Çantaya özgü ilişkiler de yazıldı (kaç bölmeli olduğu, malzemesi, nasıl kullanıldığı).

Hoca, yönlendirme yapılması gerektiğini belirtti.

> **Ders sonrası not:** Sunumda birinci kişi için oran "%66" diye söylendi; $8/12 \approx 0.667$ olduğundan doğrusu yaklaşık %67'dir. Üç oranın ortalaması da yaklaşık %72,6'dır; sunumdaki "%72" yuvarlanmış değerdir.

Sonuçlar bölümünde öğrenci, çeviri hatalarının ve Google Translate API'nin istek sınırının işi zorlaştırdığını söyledi.

7. Kelimeleri İki Boyutta Görselleştirme

Bir öğrenci, sözcükleri iki boyutlu düzlemde görselleştiren bir uygulama yaptı. Asıl amaç özellik çıkarımıydı: veriyi bir ağaç yapısında saklayıp ayıklamak, bu ağacı sözcüklerin ilişkisini gösterecek biçimde sadeleştirmek, oradan özellik vektörleri çıkarıp iki boyuta indirmek. İki boyutlu görüntü, yöntemin başarılı olup olmadığını geometrik yakınlıklara bakarak görmek içindi.

7.1 Ön işlem ve ağaç

Veride isimler, fiiller, sorular ve cevaplar var. Bu çalışmada sorular dikkate alınmadı; isim ve fiil düğümlerinin altına cevaplar, sözcük sözcük ayrılmış olarak yerleştirildi.

Cevaplar için ön işlem:

- Unicode kodlama, noktalama işaretlerini atma, küçük harfe çevirme.
- Bozuk karakterleri atma.
- Gereksiz sözcükleri atma (“olmak”, “etmek” gibi fiiller).
- Çoğul eklerini atma.
- Kodlama hatalarından kalan tek harfli parçaları atma.

İsim ve fiillere de benzer işlem uygulandı. İki sözcüklü ifadelerde ilk sözcük sıfat, ikinci isim (ya da ilk zarf, ikinci fiil) kabul edilip isim ve fiil alındı. Cevaplarda yalnız bir kez geçen sözcükler, yanlışlıkla girilmiş olabilecekleri düşüncesiyle atıldı. Sonra isim ve fiil ağaçları birleştirildi.

Ağacı sadeleştirmek için iki işlem yapıldı (ekrandan, 01:31:40):

- Çocuğu bulunmayan düğümler silindi (hiç cevabı olmayan isim ve fiiller).
- Çift bulunan düğümler birleştirildi. Örneğin isimler katmanında “araba” düğümünün altında “fren” çocuğu varsa, cevaplar katmanında başka bir düğümün altında bulunan “araba” düğümüne bu çocuk eklendi.

7.2 Özellik vektörleri ve PCA

Bütün düğümlerde en sık geçen 100 sözcük belirlendi. Elde kalan 1325 düğümün her biri için 100 boyutlu bir vektör oluşturuldu. Vektörün j’inci elemanı, o sözcüğün en sık geçen j’inci sözcükle ağaçtaki komşuluğuna göre puanlandı. Örneğin “fren”in bir üstünde “araba” varsa arabaya karşılık gelen özelliğe 2, iki üstünde “araç” varsa araca karşılık gelen özelliğe 1 eklendi. Aynı sözcük ağacın birden fazla yerinde geçerse puanlar birikir.

Hoca anlatımı netleştirmek için şu soruyu sordu:

Soru: 1325’e 100’lük matrisin satırlarında kelimeler var. Sütun numaraları ne ifade ediyor, örneğin 13. sütun? **Cevap:** 13. sütun, o kelimenin bütün kelimeler arasında en sık geçen 13. kelimeyle ilişkisinin gücünü gösterir.

Hoca bunu şöyle toparladı: matrisin (i, j) elemanı, i’inci kelimenin en sık j’inci kelimeyle ne kadar ilişkili olduğunu, yani ağaçta kaç kez komşu olduklarını gösterir.

1325×100 matris PCA ile iki boyuta indirildi (`pca.fit(X).transform(X)`) ve çizildi (ekrandan, 01:30:00). 150, 200, 300 sözcük de denendi, ama öğrencinin gözlemine göre boyut indirgeme o zaman daha kötü sonuç verdi. Haritada kakao, karabiber, irmik, çikolata, sosis, sos, sirke, şekerleme gibi yiyecekler yan yana düştü; başka bir bölgede yapı malzemeleri, bir başkasında haber, iddia, yönetmek, yaymak, savunmak; elektronik, bilgisayar, internet; yazıcı, projeksiyon. Bazı komşuluklar ise anlamsızdı (bisiklet ile tırmık, klavye ile mandal).

Hoca bunun ne sağladığını sordu; öğrenci görselleştirmeyi yalnız yöntemin başarılı olup olmadığını görmek için kullandığını söyledi. Hoca farklı boyut indirgeme tekniklerinin ya da en sık 100 yerine 500 kelimenin denenebileceğini önerdi. Öğrenci, cevapların az sayıda olduğunu, çok anlamlı sözcüklerde tek cevabın bütün anlamları yansıtmadığını, kapanmamış satırlar gibi kodlama sorunları yaşadığını ve kendi yönteminde bilgi kaybı olduğunu ekledi.

> **Ek bilgi (derste anlatılmadı):** PCA (Principal Component Analysis), verideki varyansı en çok açıklayan dik doğrultuları bulup veriyi bu doğrultulara izdüşürür. İlk iki bileşen alınınca veri iki boyutta çizilebilir. Bu dersin sonunda anlatılan LSI da aynı fikre, yani en çok bilgi taşıyan boyutları tutmaya dayanır.

8. FastText ile Kelime Temsili ve Analojiler

Bir öğrenci, veri setini FastText ile eğiterek kelimelerin anlam uzayındaki temsillerini buldu ve aralarındaki anlamsal ilişkileri, analojileri çıkaran basit bir uygulama yaptı.

Prolog dosyalarındaki kodlama sorunları temizlendikten sonra her kayıt “kömür rengi siyah” gibi arka arkaya eklenmiş bir kelime dizisine çevrildi ve bunlar birleştirilerek bir derlem (corpus) oluşturuldu. FastText’te iki model vardır: CBOW ve skip-gram. Öğrenci skip-gram’ı seçti. Gerekçesi: skip-gram tek bir kelimeden başındaki ve sonundaki kelimeleri tahmin ettiği için “kömür, rengi, siyah” gibi bir anlam ilişkisini daha iyi yakalayacağını düşünmesiydi.

Kelime analojisinin standart örneği word2vec makalesindeki “king ile queen arasındaki ilişki, man ile woman arasındaki ilişkiye eşittir” örneğidir. Veri setimizde bunun karşılığı “kömür ile siyah arasındaki ilişki, altın ile sarı arasındaki ilişkiye eşittir” olabilir; ikisi de renk ilişkisidir.

Hoca burada bir uyarı yaptı: word2vec bildiği kadarıyla “kömür-siyah, deniz-mavi” tipi analojileri çok iyi yapamıyor. Öğrencinin denemesinde “kömür:siyah :: deniz:?” sorusuna ilk sıralarda “renk, renkli, mavi” geldi; hoca bunun kötü olmadığını söyledi. Öğrenci, lemmatization yapılırsa “renk” ve “renkli”nin birleşeceğini belirtti.

En yakın komşu denemelerinde “kömür” için kül, soba, yakıt, ocak, kükürt, mazot; “altın” için gümüş, değerli, elmas, yüzük gibi kelimeler geldi.

Soru: Son gösterdiğin en yakın kelimeler word2vec değil de FastText ile eğitilmiş gibi geliyor; çünkü “altın”a en yakın kelimeler arasında “alt, altı” var. Öyle mi? **Cevap:** Evet, FastText kullanıldı.

> **Hocanın vurgusu:** word2vec “alt, altı” gibi kelimeleri bulmaz. FastText karakter düzeyinde n-gram’lara, yani alt dizgilere de baktığı için bu tür kelimeleri yakın bulur. Bu alt dizgi benzerliği FastText içinde kapatılabilir; bunun istenmediği durumlar da vardır, avantajlı tarafları da vardır.

Komut satırı uygulamasında kullanıcı bir kelime girer, model en yakın kelimeleri döndürür. Canlı denemede “kedi” için sonuçlar zayıftı (keman, kepçe, köpek, robot, kel gibi). Hoca, belki birinin “kıkırdaklı bir hayvan” gibi bir cevap yazmış olabileceğini düşündü. “Hamsi” için ham, havuç, karbonhidrat gibi kelimeler geldi. “Plastik” için elastik, lastik, silikon, poliüretan gibi iyi sonuçlar çıktı.

Hoca, sonuçları değerlendirmek için daha sistematik deneme yapılmasını önerdi; öğrenci de örneğin 10-15 analoji yazıp bunların kaçını bulduğunu raporlamanın iyi olacağını söyledi. Hoca şöyle bağladı: veri yalnız yaklaşık 500 isim ve 500 fiil için toplanmış; bu açıdan bakınca bu sonuçlar iyi bile sayılır.

> **Ek bilgi (derste anlatılmadı):** FastText, word2vec’in bir uzantısıdır. Bir kelimenin vektörü, kelimenin karakter n-gram’larının (ör. 3-6 karakterlik parçalar) vektörlerinin toplamı olarak hesaplanır. Bu sayede eğitimde görülmemiş kelimelere de vektör üretilebilir, ama ortak alt dizgisi olan kelimeler (“altın”, “altı”) anlamca ilgisiz olsa bile birbirine yakın çıkabilir.

9. Geçen Dersin Tekrarı: İstatistiksel Dil Modelleri

Aradan sonra hoca slaytlara döndü ve geçen haftayı öğrencilere sorarak hatırlattı. Geçen hafta “anlamsal uzaylar” bölümünün başında bilgi çıkarımında şablonlar ve Markov modeli (Markov varsayımı, zincir kuralı) konuşulmuştu. Bunlar, istatistiksel dil modeli denen yapıları kurmak içindi.

9.1 Dil modelinin iki temel işi

- Bir diziden sonra gelecek kelimeyi tahmin etmek.
- Bir cümlenin olasılığını bulmak.

9.2 Cümle olasılığı ne işe yarar?

Hoca bir cümlenin olasılığını bulmanın uygulama alanlarını sordu. Bir öğrenci örnek verdi: “kedi ıspanak pırasa” gibi bir kelime dizisinin Türkçe bir cümle olma olasılığı çok düşüktür, “kedi süt içti” gibi bir dizininki yüksektir. Hoca bunu “şu cümle şu cümleden daha olasıdır” bilgisine bağladı. Bu bilgi şuralarda kullanılır:

- Makine çevirisi: bir cümle için birden fazla olası çeviri varsa en olasısı seçilir.
- Yazım düzeltme: düzeltme adayları arasından en olası cümleyi veren seçilir.
- Sesten metne çeviri: ses için olası metinler arasından seçim yapılır.

9.3 Sonraki kelimeyi tahmin etmek ne işe yarar?

Metin üretmeye yarar. Bir diziden sonraki kelimeyi tahmin edebiliyorsanız onu diziye ekler, sonra bir sonrakini tahmin edersiniz; böylece metin üretilir. Geçen hafta bunun örneği yapılmıştı.

9.4 K'nin anlamı ve unigram ile trigram farkı

K, ne kadar geriye doğru bakılacağını, yani kaç önceki kelimenin kullanılacağını söyler. $K = 0$'da kelimeler bağımsızdır; $K = 1$'de bir önceki, $K = 2$'de iki önceki kelimeye bağlı geçiş olasılıkları kullanılır. Bir kelimeyi tahmin ederken K sonraki kelimeyi kullanmak da mümkündür, ama metin üretirken sonraki kelimeler henüz yoktur; standart yaklaşımda yalnız öncekiler kullanılır.

Slayttaki karşılaştırmada (ekrandan, 02:11:00) $K = 1$ ile üretilen metinlerde ardışık ikililer anlamlıydı, ama ardışık üçlüler değildi. Örneğin "%10 yılda" ifadesinde "%10" tek başına, "10 yılda" tek başına anlamlıdır, ama üçü birlikte anlamlı değildir. Trigram ($K = 2$, $t = 5$) modeliyle 1000 ekonomi haberinden üretilen cümlede ardışık üçlüler daha anlamlıdır: "yaşanan sıkıntıları için devamlı ucuz bilet satıyoruz 100 90 ı aslında biz bu bitireceğiz inşallah yıl sonuna kadar türkiyede üretilen fazlasını belirterek geçen yıl kasım ayı ihracatını". Aynı model 1000 magazin haberiyle de denenmiştir.

9.5 Bir öğrenci önerisi: geriye dönük kontrol

Soru: Metin üretirken $K = 2$ sabit kalsın; ama örneğin yedinci kelimeyi tahmin ederken beşinci kelimenin yerine oturup oturmadığını, hem önceki (üçüncü, dördüncü) hem sonraki (altıncı, yedinci) kelimelere bakarak kontrol edelim. Belirli bir eşiğin üstünde bir bozukluk varsa beşinci kelimeye dönüp cümleyi oradan yeniden üretelim. Böyle bir şey olur mu? **Cevap:** Hoca önce K'yi büyütmek gibi anladı ve K ne kadar büyükse o kadar iyi olacağını, ama K'yi büyütmenin geçen hafta konuşulan problemleri olduğunu hatırlattı. Öneri netleşince: daha önce üretilen kelimeler için, sonradan üretilen kelimeleri de kullanarak bir kontrol mekanizması kurulabilir; bu mümkündür ve daha kaliteli bir metin çıkarabilir.

10. Ardışık Geçme Yerine Aynı Dokümanda Geçme

Geçen hafta kelime frekansları hep ardışık geçme üzerinden hesaplanmıştı: w_{i-1}'den sonra w_i kaç kez gelmiş. Hoca şunu sordu: bu ardışık geçmeyi aynı dokümanda geçmeyle değiştirirsek ne olur? (ekrandan, 02:17:35)

Bunun için elde tek bir doküman değil, örneğin binlerce doküman olmalıdır. Artık "A kelimesinden sonra B kaç kez geldi" yerine "A ile B kaç dokümanda birlikte geçti" sayılır. İki kelime yan yana da olabilir, aralarında 20 kelime de olabilir; aynı dokümanda geçmeleri yeterlidir.

Burada bir önlem gerekir: frekansı çok yüksek kelimeler (stop word) atılmalıdır. Bu kelimeler bütün dokümanlarda bulunduğu için her kelimeyle çok yüksek birlikte geçme sayısı verir; atılmazlarsa her kelimeye en benzer kelimeler onlar olur.

$K = 1$ ve $t = 15$ ile (burada t üretilen dizinin uzunluğudur) üretilen cümle şudur: "faiz oranı yüksek yönetim kurumsal yönetim kurulu genel 2 satış 5 enflasyon merkez cari kredi yapı nın türk türkiyenin ancak cari büyüme merkez bankası merkez cari". Cümle üretimi açısından bu çok iyi değildir. Ama kelimelere bakınca hepsinin aynı tür dokümanlarda geçen kelimeler olduğu görülür; arka arkaya değil, aynı dokümanda geçmişlerdir. Örneğin "faiz"e en benzeyen kelimeler "merkez", "kredi", "bankası", "oranı", "cari" çıkmıştır.

Sonuç: aynı dokümanda geçme bilgisi cümle üretmek için iyi bir yol değildir, onun için ardışıklığı kullanmak daha iyidir. Ama bu yöntem benzer anlama sahip kelimeleri bulmak için kullanılabilir; ileride kelime vektörleri bulunurken bu fikir tekrar karşımıza çıkacaktır.

11. Kelime Benzerliği: Karakter ve Anlam

Kelime benzerliğinde kabaca iki tür vardır (ekrandan, 02:19:10).

11.1 Karakter benzerliği (edit distance)

Bir kelimenin başka bir kelimeye edit distance'ı, hocanın anlatımıyla o kelimeyi kaç klavye işlemiyle öteki kelimeye dönüştürebildiğimizdir. Levenshtein uzaklığı bunun en bilinen türüdür; slaytta kodları da verilmiştir (`lev.m`, `lev_sample.m`). Bununla iki kelime arasındaki uzaklık hesaplanabilir ya da bir kelimeye belli uzaklıktaki kelimeler bulunabilir.

> **Ek bilgi (derste anlatılmadı):** Levenshtein uzaklığı, bir kelimeyi ötekine dönüştürmek için gereken en az karakter ekleme, silme ve değiştirme işlemi sayısıdır. Dinamik programlamayla, iki kelimenin uzunlukları m ve n ise $O(mn)$ zamanda hesaplanır.

11.2 Anlam benzerliği

Karakter benzerliği anlam benzerliğini ölçmez. Slayttaki örnek: "Cimbom" ve "Galatasaray". Bu iki kelimenin edit distance'a göre hiçbir ortak yanı yoktur, ama anlamca birbirine benzerler. Bu anlamsal benzerliği yakalamak için başka yöntemler gerekir.

12. Saklı Anlam İndeksleme (LSI)

12.1 Fikir

Anlamsal benzerliği ortaya çıkarmak için klasik olarak kullanılan yöntemlerden biri Saklı Anlam İndeksleme'dir (Latent Semantic Indexing, LSI). Slayttaki tanımı: kelimelerin ardındaki saklı kavramları ortaya çıkarıp dokümanları bu kavramlar uzayında temsil etmek (ekrandan, 02:20:30).

İşlem aslında bir matrisi çarpanlara ayırmaktır. Derste şimdiye kadar hep doküman-terim matrisiyle (satırlarda dokümanlar, sütunlarda terimler) çalışıldı. LSI'da bunun transpozu olan terim-doküman matrisi alınır: satırlarda terimler, sütunlarda dokümanlar.

12.2 SVD ile ayrışım

Slayttaki şemaya göre $m \times n$ boyutlu A matrisi üç matrisin çarpımına ayrılır:

$$A = U \, \Sigma \, V^T$$

Burada U $m \times r$, Σ (slaytta D) $r \times r$ köşegen, V^T $r \times n$ boyutludur. Bu ayrışım, matrisin temel bileşenlerini bulmaya çalışır. Sonra her matristen bir kısım kesilir:

$$\hat{A}_k = U_k \, \Sigma_k \, V_k^T$$

U'nun ilk k sütunu ($m \times k$), Σ'nın ilk k satır ve sütunu ($k \times k$) ve V^T'nin ilk k satırı ($k \times n$) alınıp çarpılınca yine $m \times n$ boyutlu bir terim-doküman matrisi elde edilir. Fakat artık bilgi çok daha küçük boyutlu bir uzayla temsil edilmektedir.

> **Ders sonrası not:** Hoca bu ayrışımı "Single Value Decomposition" diye adlandırdı; doğru adı Singular Value Decomposition (tekil değer ayrışımı) olmalıdır; büyük olasılıkla dil sürçmesidir.

12.3 PCA ile bağlantı

Hoca bunu Temel Bileşenler Analizi (PCA) ile açıkladı. PCA'de bir matrisin bileşenleri bulunur; en baştaki bileşenler en yüksek varyansı açıklar. Verideki bilgiyi en çok tutan birinci boyuttur, sonra ikinci, sonra üçüncü gelir; en sondakiler çok az bilgi tutar. LSI'daki mantık bunun aynısıdır: U'nun son sütunlarından, köşegen matrisin son köşegen elemanlarından ve V^T'nin son satırlarından kurtulursunuz.

Bunun sağladığı şey şudur. Elinizde 5000 terim ve 100 doküman olduğunu düşünün; uzun metinler için bu normaldir. Dokümanlar 5000 boyutlu bir uzayda temsil edilmektedir ve her terim ayrı bir boyuttur. "Galatasaray", "Cimbom", "Fenerbahçe", "Sarı Kanarya", "Beşiktaş",

“Karakartal” birbirinden tamamen farklı terimlerdir. Ayrışım ve kesme yapıldıktan sonra dokümanlar belki 100 boyutlu bir uzayda temsil edilir; bu boyutlar saklı kavramlardır ve aynı anlama gelen farklı terimler aynı kavramlarda birleşir.

12.4 Küçük örnek: kedi ve köpek dokümanları

Slayttaki MATLAB örneğinde iki sınıfa ait 8 doküman vardır: ilk 4’ü kedi sınıfı, son 4’ü köpek sınıfı. Sütunlardaki 9 kelime: kedi, miyav, süt, fare, köpek, havla, kemik, hayvan, tüy (ekrandan, 02:24:40).

Doküman	Geçen kelimeler	Sınıf
1	kedi, miyav	kedi
2	kedi, süt	kedi
3	süt, hayvan	kedi
4	kedi, tüy	kedi
5	köpek, havla, tüy	köpek
6	köpek, hayvan	köpek
7	havla, kemik	köpek
8	köpek, kemik, hayvan	köpek

Hoca örnekleri sözlü olarak da okudu: bir metinde “kedi miyavladı”, birinde “kedi süt içti”, birinde kedi geçmeden “hayvan” ve “süt”, birinde “tüy” ve “kedi” geçiyor; köpek sınıfında “köpek havladı” gibi metinler var. Sınıf bilgisi tabloda yalnız okuyucu için yazılmıştır, algoritmaya verilmez.

Kod adımları (ekrandan, 02:24:40 ve 02:27:00):

```
D = D';             % matrisin kelime*dokuman olmasi gerek
[U,S,V] = svd(D);
% D = U*S*V'
% D 9*8 kelime*dokuman
% U 9*9 kelime*kavram
% S 9*8 singular values
% V 8*8 dokuman*kavram

r = 5;              % dokumanlari kac gizli kavram boyutuna indirgeyecegimiz
V = V';
yeniD = U(:,1:r)*S(1:r,1:r)*V(1:r,:);
yeniD = yeniD(1:r,:);

figure;
Z = linkage(yeniD');
dendrogram(Z);
```

SVD’ye vermeden önce matris terim-doküman hâline getirilmelidir; başta satırlarda dokümanlar, sütunlarda terimler vardır, bu yüzden transpozu alınır. Sonuçta U kelimelerin kavramlarla ilişkisini, S tekil değerleri (köşegen), V dokümanlardan kavramlara geçişi gösterir. Dokümanlar başlangıçta 9 boyutludur (9 terim). Hoca bu 9 boyutlu temsilleri SVD’ye verip ilk 5 boyutu aldığını, böylece yeni doküman temsillerini elde ettiğini ve bunları hiyerarşik kümelemeyle çizdiğini anlattı.

Tekil değerler (ekrandan, 02:27:00; python ile doğrulandı):

$$\operatorname{diag}(S) = (2.4893,\ 2.0216,\ 1.8073,\ 1.3486,\ 1.2349,\ 0.8247,\ 0.5943,\ 0.2694)$$

Dendrogramın birleşme sırası (yaprak sırası 6, 8, 5, 7, 1, 4, 2, 3):

- 6 ile 8 yaklaşık 0,16 yükseklikte birleşir.
- 5 bunlara yaklaşık 0,29’da katılır.
- 1 ile 4 yaklaşık 0,57’de birleşir.

- 7, {5, 6, 8} kümesine yaklaşık 0,88'de katılır.
- 2, {1, 4} kümesine yaklaşık 0,91'de, 3 de yaklaşık 0,96'da katılır.
- İki büyük küme yaklaşık 1,0'da birleşir.

(Birleşme yükseklikleri slayttaki koddan sonra tek bağlantılı (single linkage) kümelemeyle python'da yeniden hesaplandı ve slayttaki dendrogramla aynı çıktı.)

Hocanın vurguladığı nokta şudur: 1, 2, 3, 4 bir arada, 5, 6, 7, 8 bir arada çıkmıştır. Oysa algoritmaya yalnız D verilmiştir; ilk dört dokümanın kedi, son dördünün köpek sınıfından olduğu hiçbir şekilde söylenmemiştir. Buna rağmen 5 boyutlu uzayda aynı sınıftaki dokümanlar birbirine yakın düşmüştür.

> **Ek bilgi (derste anlatılmadı):** Koddaki `yeniD = yeniD(1:r,:)` satırı, $k = 5$ ile yeniden kurulan 9×8 terim-doküman matrisinin ilk 5 satırını, yani ilk 5 terimin (kedi, miyav, süt, fare, köpek) düzeltilmiş değerlerini alır. LSI'da dokümanların kavram uzayındaki koordinatları genellikle $\Sigma_k V_k^T$ ($k \times n$) olarak tanımlanır. İki seçenek aynı şey değildir; bu notta slayttaki kod ve dendrogram olduğu gibi aktarıldı.

Hoca, aynı örneğin daha yüksek boyutlu bir uzayda yapılmış hâlinin de slaytlarda olduğunu, orada kelimelerin koordinatlarının da rahatlıkla bulunabildiğini, kodları derste açmaya gerek olmadığını söyledi.

> **Hocanın vurgusu:** Kelime vektörlerini bulmak için illa word2vec, FastText gibi yaklaşımları kullanmak zorunda değilsiniz. Şu anda en çok kullanılanlar onlar olduğu için anılıyorlar, ama başka klasik yöntemler de vardır.

12.5 Dersin sonundaki soru

Soru: Kedi örneğinde 1'den 8'e kadar birleştirilenler dokümanlar mı? **Cevap:** Evet, dokümanlar. Dendrogramda görünen, dokümanların hiyerarşik kümelemedeki birbirine benzerliğidir. Güzel olan şu ki dokümanların sınıfına dair hiçbir bilgi verilmiyor, ama aynı sınıftakiler farklı sınıftakilere göre birbirine daha yakın çıkıyor. Bunu tamamen içlerinde geçen kelimelerle sağlıyor.

13. Sonraki Konular: Kavramsal Ağaç ve Anlamsal Uzaylar

Ders, "Kavramsal Ağaç (hiyerarşi) ile Anlamsal Benzerlik Bulma" slaytında durdu (ekrandan, 02:28:55). Bu slaytta iki temel yaklaşım yazılıdır; derste anlatılmadı, yalnız başlık olarak görüldü:

- Bağ sayma: taksonomi (kavramsal ağaç) yeterlidir.
- Ortak/müşterek bilgi (Mutual Information): taksonomi ve derlem (corpus) gereklidir.

Aradaki büyük ölçekli LSI ve Latent Dirichlet Allocation (LDA, konu modelleme) slaytlarına geçilmedi. Hoca sonraki derste anlamsal uzaylar bölümünün tamamlanacağını söyledi: önce WordNet gibi yapılarla anlamsal benzerlik bulma yöntemleri, sonra başka klasik yöntemler, sonra kelime vektörlerini bulmak için kullanılabilecek yaklaşımlar, en son da yapay sinir ağlarıyla bu işlerin nasıl yapıldığı (sinirsel dil modelleri, word2vec ve FastText'in çalışması). Bu bölümde kelimelere sabit koordinatlar veren, yani bağlamla değişmeyen anlamsal uzaylar görülecek; bağlam tabanlı dil modelleri ayrı bir slayt olarak birkaç hafta sonra gelecek.

Örnek Sorular

1. (derste sözlü, 02:08:10) Bir cümlenin olasılığını bulabilen bir dil modelinin uygulama alanları nelerdir?

> **Cevap (derste verildi):** "Şu cümle şu cümleden daha olasıdır" bilgisi, birden fazla aday arasından seçim yapılan her yerde kullanılır: makine çevirisinde olası çeviriler arasından seçim, yazım düzeltme, sesten metne çeviri.

2. (derste sözlü, 02:09:36) Bir kelime dizisinden sonraki kelimeyi tahmin etmek nerede kullanılabilir?

Cevap (derste verildi): Metin üretmede. Tahmin edilen kelime diziye eklenir ve bir sonraki kelime bu genişlemiş diziye göre tahmin edilir.

3. (derste sözlü, 02:11:00) Markov tabanlı metin üretiminde K neyi ifade eder? Metin üretirken K sonraki kelime kullanılabilir mi?

Cevap (derste verildi): K, ne kadar geriye bakılacağını, yani tahmin için kaç önceki kelimenin kullanılacağını gösterir. Bir kelimeyi tahmin ederken sonraki kelimeleri kullanmak mümkündür, ama metin üretirken sonraki kelimeler henüz üretilmediği için kullanılamaz; standart yaklaşımda yalnız öncekiler kullanılır.

4. (ekrandan, 02:17:35) Olasılık hesabında ardışık geçme yerine aynı dokümanda geçme kullanılırsa ne olur? Ne önlem gerekir?

Cevap (derste verildi): Frekansı çok yüksek kelimeler (stop word) atılmalıdır, yoksa her kelimeye en benzer kelimeler onlar olur. Üretilen dizi dilbilgisel bir cümle olmaz, birlikte geçen kelimeleri sıralar (ör. "faiz" için merkez, kredi, bankası, oranı, cari). Cümle üretimi için iyi değildir, benzer anlamlı kelimeleri bulmak için kullanılabilir.

5. (ekrandan, 02:19:10) "Cimbom" ile "Galatasaray" kelimelerinin karakter benzerliği ile anlam benzerliğini karşılaştırınız.

Çözüm (derste verilmedi): İki kelimenin (küçük harfle) hiç ortak harfi yoktur. Bu yüzden Levenshtein uzaklığı uzun kelimenin uzunluğuna eşittir: "cimbom" 6 harf, "galatasaray" 11 harf olduğundan 6 değiştirme ve 5 ekleme gerekir, uzaklık 11'dir (python ile hesaplandı). Karakter benzerliği açısından tamamen farklıdırlar, oysa anlamca aynı kulübü gösterirler. Derste vurgulandığı gibi karakter benzerliği anlam benzerliğini ölçmez.

6. (ekrandan, 02:24:40) 8 dokümanlık kedi/köpek örneğinde 8×9 doküman-terim matrisi D verilmiştir. Matris transpoz edilip SVD uygulanıyor, ilk $r = 5$ kavram boyutu alınıp dokümanlar hiyerarşik kümeleniyor. Hangi gruplar oluşur ve bu neden önemlidir?

Cevap (derste verildi, değerler python ile doğrulandı): Tekil değerler 2.4893, 2.0216, 1.8073, 1.3486, 1.2349, 0.8247, 0.5943, 0.2694'tür. Dendrogramda {6, 8, 5, 7} ve {1, 4, 2, 3} olmak üzere iki küme oluşur; bunlar köpek ve kedi sınıflarıdır. Önemli olan, algoritmaya sınıf etiketi verilmediği hâlde dokümanların yalnız içlerinde geçen kelimelere dayanarak sınıflarına göre ayrılmasıdır.

7. (derste sözlü, 01:33:49) Her satırı bir kelime olan 1325×100 boyutlu bir özellik matrisinde sütunlar en sık geçen 100 kelimeye karşılık geliyorsa (i, j) elemanı neyi ifade eder?

Cevap (derste verildi): i'inci kelimenin, en sık geçen j'inci kelimeyle ilişkisinin gücünü; bu örnekte kavram ağacında ne kadar komşu olduklarına göre verilen puanı ifade eder.

Dersin Toparlaması

Dersin ilk bölümünde birinci ödev projeleri, aynı hayat bilgisi veri tabanının ne kadar farklı biçimlerde kullanılabileceğini gösterdi: graf veri tabanında kelimeler arası yol bulma, regex ve word2vec ile bilgi çıkarımı, cümle zenginleştirme, kullanıcı oylarıyla ilişki doğrulama, WordNet ve Zemberek ile kapsayıcılık ölçme, ağaç tabanlı özellik çıkarımı ve PCA ile görselleştirme, FastText ile kelime temsilleri. Projelerin hepsi verideki gürültüyle karşılaştı. Hoca bunun gerçek veride olağan olduğunu, FastText'in alt dizgi benzerliği gibi model davranışlarının sonuçları nasıl etkilediğini ve değerlendirmenin daha sistematik yapılması gerektiğini vurguladı.

İkinci bölümde istatistiksel dil modellerinin iki temel işi (sonraki kelimeyi tahmin etme ve cümle olasılığı) hatırlatıldı. Ardışık geçme yerine aynı dokümanda geçmenin cümle üretmeye değil anlamca ilişkili kelimeleri bulmaya yaradığı gösterildi. Karakter benzerliği (edit distance) ile anlam benzerliği ayrıldı. Saklı Anlam İndeksleme ile terim-doküman matrisinin SVD ile düşük boyutlu bir kavram uzayına indirgenebileceği ve etiket verilmeden dokümanların anlamca kümelenebileceği küçük bir örnekle görüldü. Sonraki derste anlamsal uzaylar bölümüne kavramsal ağaç (WordNet) tabanlı benzerlik ölçüleriyle devam edilecek, ardından kelime vektörü yöntemlerine ve sinirsel dil modellerine geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.